An introduction to LEAN-checked arguments

A Lean + Typst sample book

Your Name

Contents

1 From definitions to proofs 3
1.1 A first definition 3
1.2 A proof with two branches 3
2 Reading the proof interactively 4
2.1 Intermediate goals 4
2.2 Asking Lean questions 4
2.3 Simple Graphs 4

1 From definitions to proofs

This book introduces a small checked argument. We first write down a proof, then look at the steps Lean uses to check it.

1.1 A first definition

Our starting point is a simple definition from the Lean project:

```
def hello : String := "Hello from Lean!"
```

Names carry their Lean types and documentation in the interactive export.

1.2 A proof with two branches

For propositions P and Q, we can prove $(P \Rightarrow P) \wedge (Q \Rightarrow Q)$ by giving one proof for each side of the conjunction.

```
theorem identity_pair (p q : Prop) :
    (p → p) ∧ (q → q) := by
  constructor
  all_goals
    intro h
    exact h
```

The two branches follow the same argument. After introducing a hypothesis, that hypothesis is already a proof of the required conclusion.

2 Reading the proof interactively

2.1 Intermediate goals

After `constructor`, Lean asks for both implications:

```
case left
p : Prop
q : Prop
⊢ p → p

case right
p : Prop
q : Prop
⊢ q → q
```

Hover over the proof-state bubbles in the previous chapter to follow the transition from these goals to a completed proof.

2.2 Asking Lean questions

We can check the theorem's type without duplicating its statement:

```
#check Demo.identity_pair
```

```
Demo.identity_pair (p q : Prop) : (p → p) ∧ (q → q)
```

Lean can also calculate the value of an expression:

```
#eval (2 + 2)
```

```
4
```

The result is 4, as expected.

2.3 Simple Graphs

Here's a simple graph in mathlib

```
#print SimpleGraph
```

```
structure SimpleGraph.{u} (V : Type u) : Type u
number of parameters: 1
fields:
  SimpleGraph.Adj : V → V → Prop
  SimpleGraph.symm : Std.Symm self.Adj := by
    aesop_graph
```

```
  SimpleGraph.loopless : Std.Irrefl self.Adj := by
    aesop_graph
constructor:
  SimpleGraph.mk.{u} {V : Type u} (Adj : V → V → Prop) (symm : Std.Symm Adj :=
by aesop_graph)
    (loopless : Std.Irrefl Adj := by aesop_graph) : SimpleGraph V
```

Here's the type of the function that gets its neighbor set for a given vertex v

```
#check SimpleGraph.neighborSet
```

```
SimpleGraph.neighborSet.{u} {V : Type u} (G : SimpleGraph V) (v : V) : Set V
```

Just randomly checking Nat's type next.

```
#check Nat
```

```
Nat : Type
```